SQL Practice Sheet

Solutions with sample output | Tables: students, courses, enrollments

Setup: Create Tables and Insert Sample Data

```
CREATE TABLE students (
  student_id INT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  age INT,
  city VARCHAR(50)
);
CREATE TABLE courses (
  course_id INT PRIMARY KEY,
  course_name VARCHAR(80) NOT NULL,
  credits INT
);
CREATE TABLE enrollments (
  enrollment_id INT PRIMARY KEY,
  student_id INT,
  course_id INT,
  marks INT,
  FOREIGN KEY (student_id) REFERENCES students(student_id),
  FOREIGN KEY (course_id) REFERENCES courses(course_id)
);

-- Insert Sample Data
INSERT INTO students VALUES
(1,'Aarav',21,'Jaipur'),
(2,'Ananya',19,'Delhi'),
(3,'Rohan',22,'Jaipur'),
(4,'Priya',20,'Mumbai'),
(5,'Karan',23,'Delhi');
INSERT INTO courses VALUES
(101,'SQL Basics',4),
(102,'Python',3),
(103,'Data Structures',5);
INSERT INTO enrollments VALUES
(1,1,101,78),(2,1,102,55),(3,2,101,88),
(4,3,103,42),(5,4,101,67),(6,2,103,30);
```

Q1: Name and city of students older than 20, sorted by name (ascending)

```
SELECT name, city
FROM students
WHERE age > 20
ORDER BY name ASC;
```

Output:

name	city
Aarav	Jaipur
Karan	Delhi
Rohan	Jaipur

Q2: Students whose name starts with the letter 'A'

```
SELECT * FROM students WHERE name LIKE 'A%';
```

Output:

student_id	name	age	city
1	Aarav	21	Jaipur
2	Ananya	19	Delhi

Q3: The 3 courses with the highest credits

```
SELECT * FROM courses ORDER BY credits DESC LIMIT 3;
```

Output:

course_id	course_name	credits
103	Data Structures	5
101	SQL Basics	4
102	Python	3

Q4: Number of students living in each city

```
SELECT city, COUNT(*) AS student_count
FROM students
GROUP BY city;
```

Output:

city	student_count
Delhi	2
Jaipur	2
Mumbai	1

Q5: Average marks per course, only courses with average greater than 60

```
SELECT course_id, AVG(marks) AS avg_marks
FROM enrollments
GROUP BY course_id
HAVING AVG(marks) > 60;
```

Output:

course_id	avg_marks
101	77.6667

Q6: INNER JOIN - student name, course name and marks

```
SELECT s.name, c.course_name, e.marks
FROM students s
INNER JOIN enrollments e ON s.student_id = e.student_id
INNER JOIN courses c ON e.course_id = c.course_id;
```

Output:

name	course_name	marks
Aarav	SQL Basics	78
Aarav	Python	55
Ananya	SQL Basics	88
Rohan	Data Structures	42
Priya	SQL Basics	67
Ananya	Data Structures	30

Q7: LEFT JOIN - students not enrolled in any course

```
SELECT s.name
FROM students s
LEFT JOIN enrollments e ON s.student_id = e.student_id
WHERE e.enrollment_id IS NULL;
```

Output:

name
Karan

Q8: Subquery - enrollments with marks above the average of all enrollments

```
SELECT student_id
FROM enrollments
WHERE marks > (SELECT AVG(marks) FROM enrollments);
```

Output:

student_id
1
2
4

Q9: UPDATE and DELETE statements

(a) Increase the credits of 'SQL Basics' by 1:

```
UPDATE courses SET credits = credits + 1 WHERE course_name = 'SQL Basics';
```

(b) Delete all enrollments where marks are below 35:

```
DELETE FROM enrollments WHERE marks < 35;
```

Courses after (a): SQL Basics now has 5 credits

course_id	course_name	credits
101	SQL Basics	5
102	Python	3
103	Data Structures	5

Enrollments after (b): enrollment 6 (marks = 30) removed

enrollment_id	student_id	course_id	marks
1	1	101	78
2	1	102	55
3	2	101	88
4	3	103	42
5	4	101	67

Q10: CREATE TABLE teachers

```
CREATE TABLE teachers (
  teacher_id INT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE,
  salary DECIMAL(10, 2) CHECK (salary > 0),
  course_id INT,
  FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
```

Constraint summary:

PRIMARY KEY on teacher_id; NOT NULL on name; UNIQUE on email; CHECK (salary > 0); FOREIGN KEY course_id references courses(course_id).